

Received January 6, 2021, accepted January 29, 2021, date of publication February 9, 2021, date of current version February 22, 2021.

Digital Object Identifier 10.1109/ACCESS.2021.3058301

Test Suite Reduction via Evolutionary Clustering

CHUNYAN XIA¹, YAN ZHANG¹, AND ZHANWEI HUI²

¹School of Computer and Information Technology, Mudanjiang Normal University, Mudanjiang 157011, China

²Institute of Evaluation and Assessment Research, Academy of Military Science, Beijing 100091, China

Corresponding author: Zhanwei Hui (hzw_1983821@163.com)

This work was supported by Heilongjiang Provincial Natural Science Foundation of China under Grant LH2020F038.

ABSTRACT Test suite reduction is an effective way to reduce the cost of regression testing by identifying and removing redundant test cases from the original test suite. In this paper, **we propose a novel cluster test suite reduction using an evolutionary multi-objective optimization algorithm.** Specifically, we use a K-means algorithm to group similar test cases to the same cluster. Then the evolutionary algorithm is used to remove redundant test cases based on the clustering results, and optimization objects are represented as the coverage-related criteria, fault-related criteria and cost-related criteria. The experimental results involving eight subject programs show that the proposed method can outperform the other three state-of-the-arts with respect to both fault detection (4.61% – 9.44%) and reduction ratio (4.10% – 10.64%). Meanwhile, the experiments also prove that our method has a better performance of missing failure rate (0.049% – 0.132%) and code coverage rate (3.34% – 6.10%). Besides, the proposed method costs are found to be comparable to the other techniques.

INDEX TERMS Regression testing, cluster analysis, genetic algorithm, test suite reduction.

I. INTRODUCTION

Software testing is an essential part of the software life cycle, which plays an important role to ensure software quality and software reliability, with the primary goal of ensuring that the deliverable software has no defects [1]. After the development of software is completed, it still needs to continuously evolve due to some reasons, such as fixing detected bugs. Regression testing has been widely adopted to ensure that software changes do not introduce new regression bugs. However, regression testing can be expensive, as the size of test suite naturally increases during software maintenance and evolution. For example, previous researches have shown that regression testing accounts for more than 80% of software testing costs [2].

Therefore, a large body of test case maintenance techniques has been proposed to reduce the overhead of regression testing, including test case prioritization, test suite reduction and test case selection [3]. Test case prioritization aims to execute the test cases more likely to detect faults as early as possible, by reordering the execution sequence of regression test cases [4]. Test case selection and test suite reduction are often used interchangeably, but they are fundamentally

different. More specifically, the goal of test case selection is to select test cases that are affected by code changes [5]. In contrast, test case reduction aims to remove redundant test cases based on satisfying redundant test requirements, thereby improving the efficiency of regression testing [6].

A number of test suite reduction techniques has been proposed in the literature [7], [8]. Most existing approaches attempt to reduce the test suite based on one or multiple testing criteria (e.g., code coverage and execution cost), which is exploited as a proxy for estimating the capability of a suite in detecting faults. Then, some reduction strategies are adopted to reduce the size of the whole test suite in a manner that does not compromise its defect detection ability [9]. However, as the size of the test suite naturally increases in regression testing, it's difficult for traditional search algorithms to find an optimal solution in such a large search space. Cluster analysis has been adopted as an efficient method in regression testing, where test cases are grouped into some clusters through establishing some similarity measures [10], [11]. It is expected that similar test cases that can detect the same bugs can be grouped into the same clusters. Thus, the test cases in the same cluster may be redundant, which can be used to design a more desired test suite reduction technique.

In this paper, we propose a multi-objective optimization method for test suite reduction based on clustering analysis.

The associate editor coordinating the review of this manuscript and approving it for publication was Jian Guo.

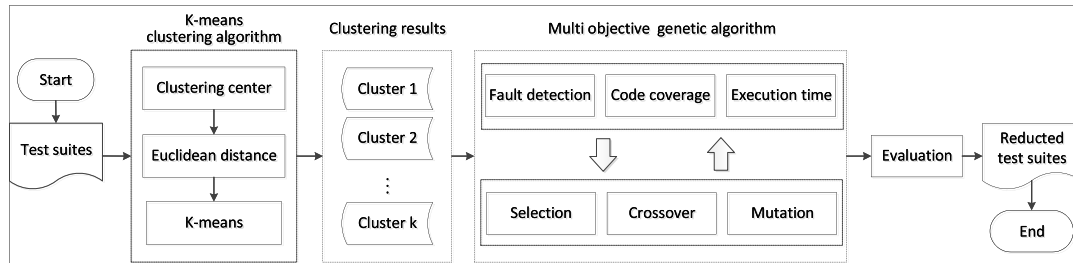


FIGURE 1. Framework of the proposed method.

Specifically, a K-means algorithm is utilized to cluster test cases so that test cases assigned to the same cluster have high similarity. Then, a multi-objective genetic algorithm is adopted to remove redundant test cases from the clustering results. We also conduct a series of real-life programs to evaluate the effectiveness and efficiency of the proposed method.

The rest of this paper is organized as follows: Section II introduces some background information. Section III describes the proposed method in detail. Section IV explains details of the empirical study and provides the detailed results of the experiment. Section V discusses some related work, and Section VI summarizes the paper, including highlighting some potential future work.

II. BACKGROUND

A. CLUSTER ANALYSIS

Cluster analysis is the task of grouping a set of objects based on the similarity of their hidden features (e.g., execution profiles [12]). Thus, objects in the same group (i.e., a cluster) are more similar (in some sense) to each other than to those in other groups. In other words, the greater the similarity of objects within one group and the diversity of those between different groups, the better the cluster technique will perform.

Clustering techniques can be roughly distinguished as hard clustering and soft clustering, where within the former an object can be shared by different clusters, and within the latter an object belongs at most to one cluster. For example, the K-means, which is one of the most popular hard clustering techniques, creates a single-level division of data objects based on the centroid-based clustering technology. Meanwhile, it has been widely applied in various regression testing activities, including test case prioritization, test case selection and test suite reduction [11], [13], [14]. To that end, K-means firstly estimate the number of clusters, k , according to the total number of objects. Secondly, k objects are selected randomly as the initial centroid of the clusters. Thirdly, each object is assigned to the nearest cluster centre after the distance of each object and the centroid of each cluster is calculated. The centroid of each cluster is then updated based on the current clustering results. The clustering process continues until the centroid of each cluster does not change. The basic steps are presented in Table 1.

TABLE 1. Basic K-means algorithm.

Basic K-means algorithm	
1	select k objects as the initial centroid of the cluster randomly (k is the number of expected clusters)
2	repeat
3	calculate the distance between each object and the centroid of each cluster and assign it to the nearest cluster
4	update the centroid of each cluster
5	until
6	the centroid does not change

In order to assign objects to the nearest cluster, a proximity measure is employed to quantify the “nearest” concept of the data. The similarity measure used by the K-means algorithm is relatively simple, since the algorithm only repeatedly calculates the similarity from each object to each centroid. Typically, Euclidean distance is used to measure its proximity. For low-dimensional Euclidean space data, many similarity calculations can be avoided, which can significantly speed up the K-means algorithm. Therefore, in this paper, we use Euclidean distance to measure the similarity of objects.

Assuming that the two points in Euclidean space are $x = (x_1, x_2, \dots, x_n)$ and $y = (y_1, y_2, \dots, y_n)$, the formula for calculating the Euclidean distance between them is:

$$\text{dist}(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2} \quad (1)$$

B. MULTI-OBJECTIVE OPTIMIZATION

Multi-objective optimization is an area of multiple criteria decision making, which involves more than one objective function to be optimized at the same time. Various multi-objective optimization algorithms have been extensively adopted in many fields of software testing, including test suite reduction where optimal decisions need to be taken in the presence of trade-offs between two or more conflicting objectives (e.g., code coverage and execution time). NSGA-II (Non-dominated Sorting Genetic Algorithm II) with elite strategy is one of the most popular and widely used classical multi-objective optimization algorithms.

A general description of multi-objective optimization problems can be defined as follows. Given the decision variables $x = (x_1, x_2, \dots, x_n)$, two constraints should

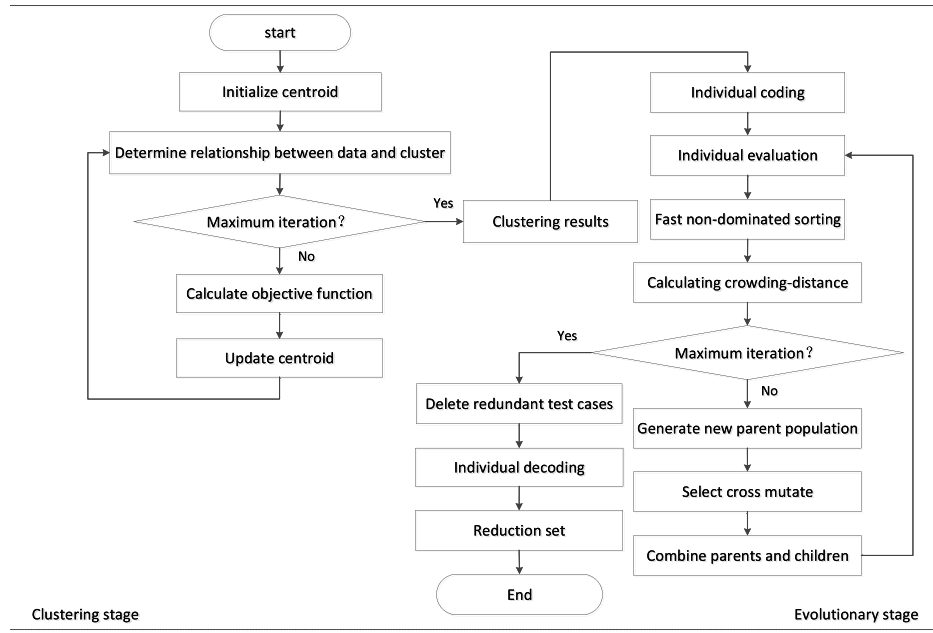


FIGURE 2. Flow chart of the proposed method.

be satisfied:

$$\begin{cases} g_i(x) \geq 0 & (i = 1, 2, \dots, k) \\ h_i(x) = 0 & (i = 1, 2, \dots, l) \end{cases} \quad (2)$$

Considering m conflicting sub-objectives, the optimization goal can be formulated as follows.

$$f(x) = \max(f_1(x), f_2(x), \dots, f_m(x)) \quad (3)$$

The solutions satisfying the constraints and variable bounds constitute a feasible decision variable space Ω . Then the multi-objective optimization problem can be transformed into the problem of finding the optimal solution set of Pareto.

$$P^* = \{x^* \in \Omega \mid \nexists x \in \Omega, f_i(x^*) \geq f_i(x) (i = 1, 2, \dots, m)\} \quad (4)$$

C. FRAMEWORK

Figure 1 presents the framework of proposed method. Firstly, the test cases are clustered using the K-means algorithm. The goal of clustering analysis is to assign the original test cases to the different cluster according to similarity, which mainly involves the calculation process of distance measurement and the cluster centroid. Then, the multi-objective genetic algorithm is used to reduce the test suite, which aims to optimize the clustered test cases according to multiple optimization goals. This process involves the design strategy of optimization goals, operating operators in genetic algorithms. Finally, an empirical study is conducted to evaluate the proposed method.

III. APPROACH

The evolutionary-based cluster test reduction method involves two parts: cluster analysis and evolution algorithm. The flowchart of the proposed method is shown in Figure 2. In the clustering stage, a simple K-means algorithm is adopted. Specifically, K initial centroids are randomly selected, and the proximity is measured using Euclidean distance to determine the affiliation of data points and clusters. Subsequently, the value of the objective function is calculated to update the cluster's centroid. The above process is repeated until the maximum number of iterations is reached to obtain the clustering results. In the evolutionary stage, the evolution method utilizes a multi-objective genetic algorithm. In advance, the clustering results are coded for individuals to evaluate their priority. Then, the non-dominated genetic algorithm is used to sort the evolved individuals, and the individuals at the same level calculate the degree of congestion. Subsequently, genetic algorithm selection, crossover, and mutation operations are performed to evaluate new individuals. Finally, redundant test cases are removed based on the principle of overriding all coding, and individuals are decoded to output a reduced set of test suites.

A. K-MEANS ALGORITHM

Various techniques have been proposed to perform clustering analysis in literature [15], including hierarchical clustering, k-means clustering and density-based clustering. Following previous work [11], [13], the k-means clustering is adopted in our work, which is widely adopted in regression testing research. Specifically, K test cases are randomly selected as the initial centroid of each cluster. Then the Sum of the Squared Error (denoted as SSE) is used as the objective

function of the clustering algorithm, as the distance of two test cases is measured by Euclidean distance. Specifically, we calculate the error of each test case (i.e., Euclidean distance to the nearest centroid), and then get the sum of squares of the error. The formula of SSE is defined as follows:

$$SSE = \sum_{i=1}^K \sum_{x \in C_i} \text{dist}(c_i, x)^2 \quad (5)$$

where, K represents the number of clusters, C_i represents the i -th cluster, c_i represents the centroid of cluster C_i , x represents the point in the cluster, and $\text{dist}(c_i, x)^2$ represents the Euclidean distance between two points (i.e., x and c_i).

According to several existing studies [10], [11], most existing cluster techniques are based on execution profiles of the test suite. Typically, each test case is represented by a feature vector of function call profile in cluster analysis, and the process may be time-consuming. In fact, a string of size n can be seen as a vector of characters in an n -dimensional space, and the characters can be associated with their ASCII code (or any other digital code). It is thus possible to compute the classical distances in an n -dimensional space (e.g., Euclidean distances). Thus, we perform a static cluster analysis, where each test case is represented as a string input:

$$\text{dist}(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (6)$$

where x and y are the string of test cases to compare. It is worth noting that the shortest one is filled with $\text{char}(0)$, when strings have different lengths.

B. OPTIMIZATION OBJECTIVES

Existing studies have proposed various testing criteria to optimize existing test cases, including coverage-related criteria [16]–[18], fault-related criteria [19], [20] and cost-related criteria [21]. For an overview of existing work, we refer to [22], [23]. Mukherjee *et al.* [22] present a systematic mapping study of test case optimization techniques over the period 2001–2018. Therefore, according to existing work, the three most commonly used testing criteria, fault detection rate, statement coverage, and execution time, are chosen as optimization objectives.

Coverage-related criteria is represented by average of the percentage of statement coverage (APSC), which can measure the rate at which a prioritized test suite covers the statements. Specifically, Statement coverage is defined as the percentage of statements covered by a test case, which is the widely used structural coverage criterion. It can measure to what extent a test case covers statements during test-case execution. Higher statement coverage indicates larger fault-detection capability because without covering faulty statements a test case cannot reveal the corresponding faults. Statement coverage Objective aims to use the least number of test cases for achieving maximum value of covered statements. It is defined as the ratio between the number of

covered statements and the number of all executable statements when the test case executes the program.

Given a test suite T containing n test cases that covers a set of m_1 statements, the calculation formula is as follows:

$$APSC = 1 - \frac{1}{m_1 \times n} \sum_{i=1}^{m_1} TS_i + \frac{1}{2n} \quad (7)$$

where TS_i represents the first test case in the order T that covers statement i .

Fault-related criteria is represented by average of the percentage of faults detected (APFD), which can measure the rate at which a prioritized test suite can detect faults. Higher APFD indicates larger fault-detection capability. Fault detection rate is mainly used to calculate the average cumulative proportion of defects detected during the execution of test cases.

Given a test suite T containing n test cases that detects a set of m_2 faults, the calculation formula is as follows: The calculation formula is as follows:

$$APFD = 1 - \frac{1}{m_2 \times n} \sum_{i=1}^{m_2} TF_i + \frac{1}{2n} \quad (8)$$

where TS_i represents the first test case in the order T that detects fault i .

Cost-related criteria is represented as the execution time of test cases, which can measure how long do the test case take. The execution time, mainly used to calculate the execution time when the fault detection rate and statement coverage of the test case reaches the maximum, is calculated as follows. Given a test suite T containing n test cases and ET_i represents the execution time of i th test case, the calculation formula is as follows:

$$EET = \sum_{i=1}^m ET_i \quad (9)$$

C. GENETIC ALGORITHM

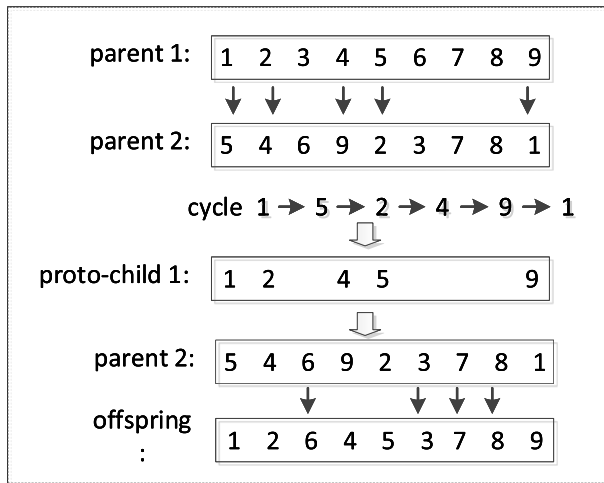
The genetic algorithm is a computational model that simulates the evolutionary process of natural organisms by targeting individuals in a population. Meanwhile, it is an efficient evolutionary method for global search of optimal solutions [24]. In the multi-objective evolution method of test suite reduction, the initial population of the genetic algorithm is the arrangement of the test cases in the clustering result. In addition, the individual coding method adopts the decimal encoding method, and the basic gene position is the sequence number of the test case in the clustering result.

Selection. The selection operation uses the tournament selection method. More specifically, for each generation, a number of $n/2$ individuals are randomly selected from the n population. The best individual is then selected to composite the offspring population-based on evaluation. The above operations are repeated until the new population size reaches n .

Crossover. The crossover operation uses a circular crossover method, as shown in Figure 3. Firstly, the algorithm

TABLE 2. The subjects of experiment.

Program Name	Description	Version	LOC	#Fault	#Test
<i>tot_info</i>	information measure	2.0	565	23	1052
<i>tcas</i>	altitude separation	2.0	173	41	1608
<i>schedule</i>	priority scheduler	2.0	412	9	2650
<i>schedule2</i>	priority scheduler	2.0	374	10	2710
<i>print_tokens</i>	lexical analyzer	2.0	726	7	4130
<i>print_tokens2</i>	lexical analyzer	2.0	570	10	4115
<i>replace</i>	pattern replace	2.1	564	32	5542
<i>space</i>	European Space Agency program	2.0	6199	35	13585

**FIGURE 3.** Cyclic crossover process.

randomly selects a gene on parent 1, find the gene number of the corresponding location of parent 2, and then returns the gene position with the same number in parent 1. Secondly, repeat the step 1 process until the identified gene forms a ring. Thirdly, the identified genes in parent 1 are selected to generate offspring and ensure that the positions correspond. Lastly, the remaining genes from parent 2 are put into the offspring.

Mutation. The mutation operation uses the sequential mutation method. More specifically, a gene from the parent individual is randomly selected and the test case for that point is removed. A test case that does not duplicate a gene already present in the current individual is then randomly inserted to form a new offspring individual.

IV. EMPIRICAL STUDY

This section describes the family of empirical studies performed, including the research questions, subjects, evaluation metrics, and results.

A. RESEARCH QUESTIONS

In order to evaluate the performance of the proposed method, we aim to explore the following research questions.

RQ1 How effective is the proposed method to reduce the test suite compared with other approaches?

RQ2 How effective is the proposed method to reduce test cases compared with other approaches?

RQ3 How does the efficiency of the proposed method compare with other approaches?

B. SUBJECT PROGRAMS, TEST SUITES AND FAULTS

We conduct our study on 7 Siemens programs (originally created by Tom Ostrand and colleagues at Siemens Corporate Research) and 1 European Space Agency program (European Space Agency program). These programs are widely used as benchmarks in regression testing, including test case reduction [10], [12]. All these programs and their test suites are taken from Software-artifact Infrastructure Repository (SIR).¹

Our experiment requires programs with varying numbers of faults. For each program, multiple faulty versions are created by manual injection of faults, usually by modification of a single line of code in the program. Thus, each program is provided with a correct base version and multiple single-fault version, resulting in 175 program versions in our experiment. Table 2 summarizes the subject program details, with Columns 1 to 6 presenting the program name, a brief description of the program, version, size, number of faults and test cases, respectively.

C. EVALUATION METRICS

In a multi-objective optimization scenario, multiple objectives are considered simultaneously. Thus, following existing work [25], to evaluate the outcome of optimization algorithm, several metrics are adopted, including reduction rate, code coverage, fault detection and execution time.

To answer RQ1, we consider two metrics to measure the effectiveness of the reduced test suite, namely reduction rate of the test suite and failure detection rate of the test suite.

Reduction rate of the test suite is calculated by the ratio of the reduced test suite size to the original test suite size. The calculation formula is as follows:

$$TestSuite_{Reduction} = \frac{|Testsuite| - |Testsuite_{Red}|}{|Testsuite|} \times 100\% \quad (10)$$

where, $|TestSuite|$ represents the number of the original test suite, and $|TestSuite_{Red}|$ represents the number of the reduced test suite.

¹<https://sir.csc.ncsu.edu/content/sir.php>

Fault detection rate of the test suite is calculated by the ratio of the number of faults detected by the reduced test suite to the number of total faults detected by the original test suite. The calculation formula is as follows:

$$TestSuite_{FaultDetection} = \frac{|FaultDetection_{Red}|}{|FaultDetection|} \times 100\% \quad (11)$$

where, $|FaultDetection|$ represents the number of faults detected by the original test suite, and $|FaultDetection_{Red}|$ represents the number of faults detected by the reduced test suite.

To answer RQ2, we consider two metrics to measure the effectiveness of reduced test cases, namely average missing faults rate of test cases and the average code coverage of test cases.

Average missing fault rate of test cases is the ratio of the number of undetected faults to the number of executed test cases. The formula is as follows:

$$TestCase_{FaultDetection} = \frac{|FaultDetection - FaultDetection_{Red}|}{|Executioncase|} \times 100\% \quad (12)$$

where, $|FaultDetection|$ represents the number of faults detected by the original test suite, $|ExecutionCase|$ is the number of executed test cases, and $|FaultDetection_{Red}|$ represents the number of faults detected by the reduced test case.

Average code coverage of a test case is the ratio of the number of statements covered to the number of executed test cases. The formula is as follows:

$$TestCase_{coverageCode} = \frac{|Coveragecode|}{|Executioncase|} \times 100\% \quad (13)$$

where, $|ExecutionCase|$ represents the number of recuted test cases, and $|CoverageCode|$ represents the number of codes covered by the executed test cases.

To answer RQ3, we consider the execution time of test suite reduction, which is mainly divided into two parts (i.e., clustering time and evolution time). The formula is as follows:

$$ExecutionTime = ClusterTime + EvolutionTime \quad (14)$$

where $ClusterTime$ represents the time required for test suite clustering, and $EvolutionTime$ represents the time required for test suite evolution.

D. EXPERIMENTAL SETUP

In this paper, we propose an evolutionary method for test suite reduction based on clustering analysis (TSR_CE). TSR_CE uses the K-means algorithm to cluster test suite while employing a multi-objective genetic algorithm for test suite reduction with statement coverage, fault detection rate, and execution time as optimization targets.

We primarily consider several representative methods, which have similar clustering techniques and optimization

goals to our method, as experimental comparison. Literature [26] proposed a multi-objective test suite reduction technology (TSR_LF) based on linear formulas, where the optimization targets are statement coverage, fault detection capability, and test execution time. Literature [27] proposed a test suite reduction algorithm (TSR_FK) based on fuzzy K-means, which aims to minimize the test case reduction while maintaining the coverage and error detection rate of the reduced test suite. Meanwhile, we also propose a traditional multi-objective evolution method of test suite reduction (TSR_T), where statement coverage, fault detection rate, and test execution time are used as optimization targets. Therefore, the above three test suite reduction methods (i.e., TSR_LF, TSR_FK and TSR_T) are suitable for comparison with the proposed methods (TSR_CE), as they can represent state-of-the-arts and have similar adopted techniques to our method.

In the K-means clustering algorithm, the number of initial centroids K is the main input parameter. If the selected K value is too large, some similar test cases may be clustered into different clusters. In contrast, if the selected K value is too small, some different test cases may be clustered into the same cluster. Therefore, inappropriate K values may produce low-quality clusters, resulting in poor test suite reduction. This paper compares the results of multiple experiments and finally determines that the number of initial centroids is 5% of the number of test cases. The experimental results are basically consistent with that of the paper [28], which is also based on the clustering method, further indicating that the setting of K value in the clustering algorithm is reasonable. In the genetic algorithm, the initial population is a randomly selected from the clustering results with the population size equal to 10% of the size of clustering results, and the maximum number of iterations is 100,000. Furthermore, each tested program is run 30 times to ensure the reliability of the results, and the average value of the experimental data is taken as the experimental result. It should be noted that taking test case reduction as a multi-objective optimization problem may generate multiple solutions in the Pareto optimal solution set. Then we used a random selection method for selecting an element.

E. RESULTS AND ANALYSIS

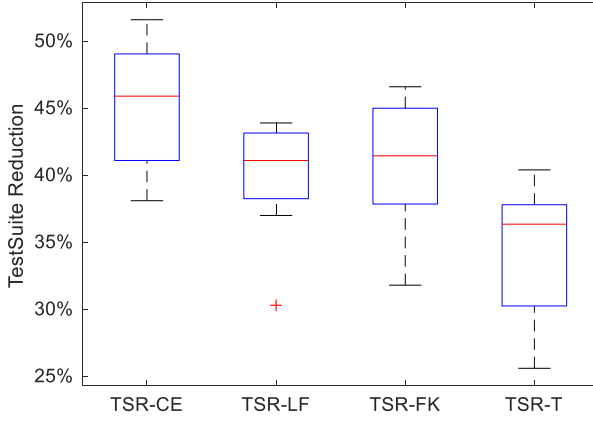
1) THE EFFECTIVENESS OF PROPOSED METHOD TO REDUCE TEST SUITE

To answer RQ1, Table 3 provides comparisons of the reduction rate of the test suite and the failure detection rate of the test suite for the different techniques. And Figure 5 presents box plots of the distribution of the experimental results. Each box plot shows the mean (square in the box), median (line in the box), and upper and lower quartiles (25th and 75th percentile) for the values for the techniques.

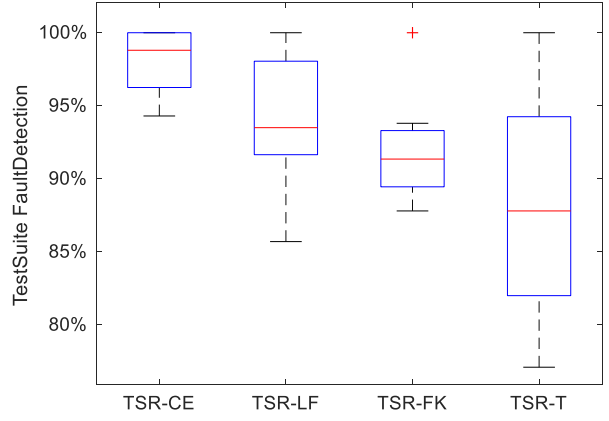
Test suite reduction rate. In the TSR_CE method, the reduction rate of print_tokens is 45.3%, and it is 45.6% for the TSR_Fk method. Compared with the TSR_Fk method,

TABLE 3. The experimental results of the test suite.

Program	Test Case Reduction (%)				Test Suite Fault Detection (%)			
	TSR_CE	TSR_LF	TSR_FK	TSR_T	TSR_CE	TSR_LF	TSR_FK	TSR_T
tot_info	38.1	30.3	31.8	25.6	100	100	91.3	86.9
tcas	46.5	39.5	43.7	28.7	97.6	92.7	87.8	85.4
schedule	48.3	42.6	39.2	36.9	100	100	88.9	100
schedule2	51.6	43.7	44.4	40.4	100	93.3	90.0	97.3
print_tokens	45.3	40.6	45.6	37.2	95.7	85.7	92.8	78.6
print_tokens2	43.1	41.6	36.6	38.4	100	96.1	100	91.2
replace	49.8	43.9	46.6	35.8	96.8	90.6	93.8	88.7
space	39.1	37	38.9	31.8	94.3	93.7	91.4	77.1
means	46.24	41.27	42.14	35.6	97.77	93.16	92.10	88.33



(a) Test Suite Reduction



(b) Test Suite Fault Detection

FIGURE 4. The effectiveness of test suite.

the TSR_CE method only has a slightly lower reduction rate for print_tokens, but the difference is only 0.3%. Compared with other methods, the TSR_CE method has the highest average reduction rate. The test suite reduction rate of various methods is shown in Figure 4, which also answers the problem 1. Our method can effectively reduce the size of the test suite and remove redundant test cases.

Test suite failure detection rate. Among all methods, the average failure detection rate of the TSR_CE method is 97.77%, which can detect almost all errors in the program. The failure detection rate of various methods is shown in Figure 4(b), which also answers the problem 1. Our method can effectively improve the failure detection rate of the test suite and can find more errors in the tested program.

In order to further verify the effectiveness of the proposed method in this paper, we also adopt statistical analysis to confirm whether they have significant differences or not in the test suite reduction rate result. The hypothesis test method used in our work is Z test, and we set the significance level $\alpha = 0.05$, then $Z_{\alpha/2} = 1.96$. The hypothesis tested is that there is no significant difference in the test suite reduction rate between the comparison methods.

Suppose the random variable X represents the average reduction rate of the test suite, and it approximately obeys the normal distribution $X \sim (\mu, \sigma^2)$. Thus, we compare the size

of the mean μ of random variables in each method. The larger the value, the more test cases the method reduces, indicating better effectiveness. Considering that the sample variance is an unbiased estimate of the population variance, the sample variance is used as the estimated value of the population variance. Particularly, sample size $n_1 = n_2 = n_3 = n_4 = 30$, sample mean $\bar{X}_1 = 46.24$, $\bar{X}_2 = 41.27$, $\bar{X}_3 = 42.14$, $\bar{X}_4 = 35.6$, and sample variance $\sigma_1^2 = 8.93$, $\sigma_2^2 = 12.31$, $\sigma_3^2 = 11.88$, $\sigma_4^2 = 11.38$.

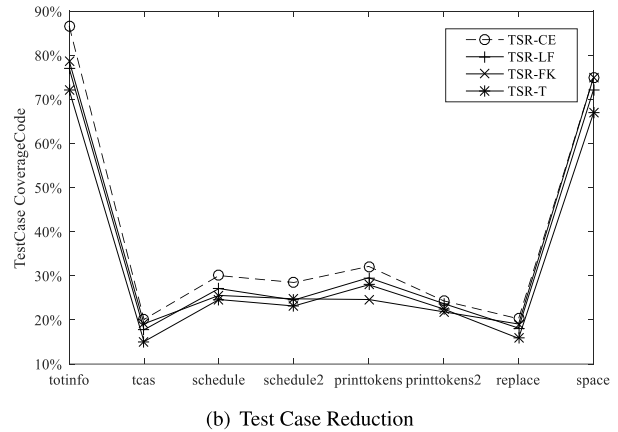
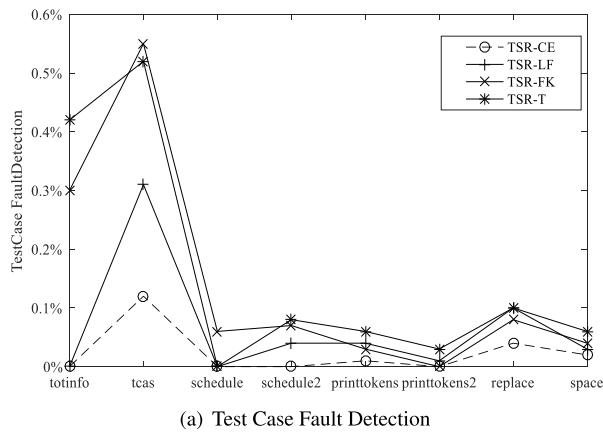
To that end, we (1) establish the null hypothesis $H_0 : \mu_1 \leq \mu_2, H_0 : \mu_1 \leq \mu_3, H_0 : \mu_1 \leq \mu_4$, and alternative hypotheses $H_1 : \mu_1 > \mu_2, H_1 : \mu_1 > \mu_3, H_1 : \mu_1 > \mu_4$; (2) construction statistics $Z_1 = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}}$, $Z_2 = \frac{\bar{X}_1 - \bar{X}_3}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_3^2}{n_3}}}$, $Z_3 = \frac{\bar{X}_1 - \bar{X}_4}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_4^2}{n_4}}}$;

(3) consider the rejection domain, $Z_1 \geq Z_{\alpha/2}, Z_2 \geq Z_{\alpha/2}, Z_3 \geq Z_{\alpha/2}$; (4) calculate the value of the statistic, namely $Z_1 = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}} = \frac{46.24 - 41.27}{\sqrt{\frac{8.93}{30} + \frac{12.31}{30}}} \approx 5.90$, $Z_2 = \frac{\bar{X}_1 - \bar{X}_3}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_3^2}{n_3}}} = \frac{46.24 - 42.14}{\sqrt{\frac{8.93}{30} + \frac{11.88}{30}}} \approx 4.92$, $Z_3 = \frac{\bar{X}_1 - \bar{X}_4}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_4^2}{n_4}}} = \frac{46.24 - 35.6}{\sqrt{\frac{8.93}{30} + \frac{11.38}{30}}} \approx 12.93$;

(5) draw the conclusion: $Z_1 \geq Z_{\alpha/2}, Z_2 \geq Z_{\alpha/2}, Z_3 \geq Z_{\alpha/2}$. Therefore, reject the null hypothesis: $H_0 : \mu_1 \leq \mu_2, H_0 : \mu_1 \leq \mu_3, H_0 : \mu_1 \leq \mu_4$, and accept alternative hypothesis: $H_1 : \mu_1 > \mu_2, H_1 : \mu_1 > \mu_3, H_1 : \mu_1 > \mu_4$. It can be

TABLE 4. The experimental results of test cases.

Program	Test Case Fault Detection (%)				Test Case Coverage Code (%)			
	TSR_CE	TSR_LF	TSR_FK	TSR_T	TSR_CE	TSR_LF	TSR_FK	TSR_T
tot_info	0.00	0.00	0.30	0.42	86.7	77.08	78.80	72.16
tcas	0.12	0.31	0.55	0.52	20.12	17.78	19.14	15.10
schedule	0.00	0.00	0.06	0.00	30.07	27.12	25.57	24.64
schedule2	0.00	0.04	0.07	0.08	28.51	24.52	24.80	23.17
print_tokens	0.01	0.04	0.03	0.06	32.15	29.58	24.64	27.99
print_tokens2	0.00	0.01	0.00	0.03	24.34	23.72	21.85	22.49
replace	0.04	0.10	0.08	0.10	20.27	18.16	19.08	15.84
space	0.02	0.03	0.04	0.06	74.98	72.41	74.68	66.91
means	0.027	0.076	0.119	0.159	39.64	36.30	36.07	33.54

**FIGURE 5.** The effectiveness of test suite.

seen that the comparison results are significantly different. In other words, the average reduction rate of the test suite of the TSR_CE method is significantly higher than that of TSR_LF, TSR_FK and TSR_T.

2) THE EFFECTIVENESS OF PROPOSED METHOD TO REDUCE TEST CASE

To answer RQ2, Table 4 shows the average missing failure rate and average code coverage of test cases in the TSR_CE, TSR_LF, TSR_FK, and TSR_T. The average missing failure rate of test case refers to the ratio of the average error that is not detected for each test case. The smaller the value, the better the algorithm. When the value is 0, it means that all failures of the program can be detected. The average code coverage of test case refers to the average number of code lines that can be covered by each test case. The larger the value, the better the algorithm, indicating that the average number of code lines covered by each test case is greater.

The average missing failure rate of test cases. In the TSR_CE method, the average failure rate of tcas, print_tokens, replace and space are not 0, but the average result of all tested programs is only 0.027%, which is almost negligible. The final results of various methods are shown in Figure 5, which also answers the problem 2, i.e., the reduced test suite obtained by our method has a high ability to detect failures and will not miss many failures.

TABLE 5. The efficiency results of experiment.

Program	Execution Time (s)			
	TSR_CE	TSR_LF	TSR_FK	TSR_T
tot_info	6.4	6.1	6.3	6.8
tcas	13.5	14.4	13.1	15.2
schedule	13.1	13.7	12.8	14.6
schedule2	15.7	16.4	15.9	17.5
print_tokens	145.2	150.5	140.7	161.3
print_tokens2	133.6	141.0	135.8	152.4
replace	304.4	323.6	315.3	331.9
space	1402.1	1493.3	1353.2	1581.7
means	254.25	269.88	249.14	285.18

The average code coverage of test cases. Among all the methods, the TSR_CE method has the highest average code coverage. The code coverage results of various methods are shown in Figure 5(b), which also answers problem 2, i.e., the reduced test suite obtained through our method has a higher ability to cover code by test cases, which can better improve the quality of the program.

3) THE EFFICIENCY OF PROPOSED METHOD

To answer RQ3, Table 5 shows the execution time of the test suite reduction for the four methods TSR_CE, TSR_LF, TSR_FK and TSR_T. Among them, the average execution time of the TSR_FK method is the shortest. However, the average execution time of the TSR_CE method is only

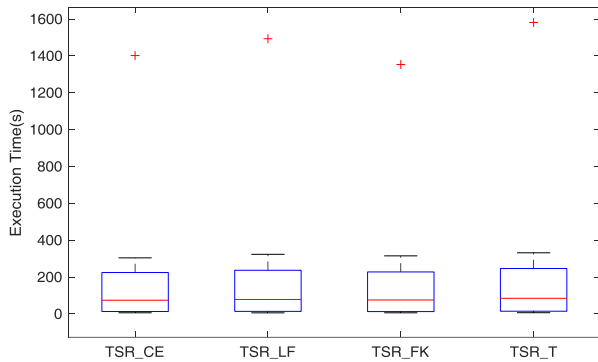


FIGURE 6. Execution Time.

5.11 seconds longer than it. The execution time of the test suite reduction for the four methods is shown in Figure 6. It can be seen that their execution time differs slightly. In other words, our method has great execution efficiency with high test suite reduction rate and fault detection rate.

V. RELATED WORK

Regression testing is the process of testing modified software to ensure its quality. Test case prioritization, test case selection, and test suite reduction are three techniques that focus on reducing test costs by reusing existing test cases. Since the focus of these methods is different, they belong to different technologies. In this paper, we propose a novel method of test suite reduction based on cluster analysis and Multi-objective Optimization, and we will focus on techniques related to the above aspects.

Traditional methods. In traditional methods, researchers have proposed many techniques and methods regarding test suite reduction [29]. Most existing methods use information based on code coverage, applications, execution costs, and state models. For example, Harder *et al.* [30] and McMaster *et al.* [31] used dynamic heuristics to reduce the size of test suites and conducted empirical evaluations. Campos *et al.* [32] encoded the relationship between test cases and test requirements into the coverage matrix, and then derive a set of constraints to calculate the optimal reduction set. Gotlieb *et al.* [33] proposed a test suite reduction method based on requirement coverage. The method composes a flow network through a given test suite and the requirements covered by it, and applies constraint programming technology to search for the optimal flow. Arlt *et al.* [34] proposed a static analysis method based on program slicing to reduce the number of test cases. Lin *et al.* [35] collect code execution costs by running test cases, and identify test cases that can reduce execution costs. Gotlieb *et al.* [36] used minimization techniques to identify and eliminate redundant test cases from the entire test suite to reduce the execution cost of test cases. Mussa *et al.* [37] proposed a model-based framework to achieve reuse and optimization through different test levels, and remove redundant test cases.

A recent trend related to regression testing is to reduce test cases while maintaining its fault detection capabilities. For example, Groce *et al.* [38] proposed a method to keep the coverage of the original test set to reduce test cases. Alipour *et al.* [7] proposed a test case simplification method to maintain fault detection capabilities while allowing it to retain only partial code coverage.

Most traditional methods only focus on a certain aspect of information, namely, single-objective optimization is the mainstay, and then use heuristic criteria to remove redundant test cases from different perspectives. The method proposed in this paper is an evolutionary algorithm. When reducing the test suite, it also considers the balance between multiple objectives, which allows us to better explore the solution space.

Cluster analysis. Cluster analysis uses some distance measures and clustering algorithms to group tests into some clusters, which is widely used in test suite reduction and test case selection. For example, Zhang *et al.* [10] presented a new regression test selection technique by clustering the execution profiles of modification traversing test cases. Meanwhile, Chen *et al.* [11] adopted a semi-supervised clustering method, namely semi-supervised Kmeans (SSKM), to improve cluster test selection. Instead of clustering test cases based on the similarity of their execution profiles only once, Wang *et al.* [12] conduct more than one iteration of clustering using weighted execution profiles.

Multi-objective approach. With the increasing complexity of software programs, single-objective test case optimization technology can no longer meet the needs of regression testing. Thus, the application of multi-objective technology has become an inevitable trend. Mirarab *et al.* [29] proposed a method to reduce test cases based on two criterion functions by encoding the selection of test cases as an integer linear programming problem. Shi *et al.* [20] proposed a method to reduce test cases by considering the two goals of code coverage and the number of killed variants. Most of the existing multi-objective test suite reduction methods are based on evolutionary algorithms, usually considering code and requirement coverage information, and trying to strike a balance between this information and the execution cost of test cases. For example, Ma *et al.* [39] designed an objective function that combines code coverage and execution cost to reduce test suites. Yoo and Harman [8] propose a test suite reduction method based on the Pareto optimal solution, in which coverage and fault detection information are explicitly considered. Gupta *et al.* [40] proposed a multi-objective method based on code and mutation coverage to generate a minimal test suite that has both the ability to detect and locate faults. The above-mentioned multi-objective method redefines the multi-objective problem as a single objective by using an optimization function (reference [39]), or reduces the test suite containing only two optimization objectives (reference [8], [40]).

Recently, Costa [41] proposed a test suite reduction method based on three goals (including code coverage, execution

time, and fault location capabilities) to reduce test cases. Rzener *et al.* [26] proposed a linear formula reduction test suite method based on sentence coverage, fault detection rate, and execution time. Experiments show that the optimization method based on the three objectives can better reduce the test suite. Gao *et al.* [27] proposed a test case set reduction algorithm based on fuzzy K-means, which aims to find similar test cases and maximize the simplified test case set. Cheng *et al.* [42] proposed a semi-supervised clustering method to mine the hidden paired constraint information from the historical execution records of regression testing, and optimize the selection results of test cases. Recently, more and more researchers have tried to improve retrospective testing techniques by combining technologies in other fields such as data mining, machine learning, and information retrieval. It is expected that this trend will continue with increasing interest [43]. Inspired by the above work, we propose an evolutionary method for test suite reduction based on clustering analysis. The K-means algorithm is used to cluster test cases as well as the failure detection rate, statement coverage, and execution time as the optimization goals. Finally, the test suite reduction is realized through a multi-objective evolution algorithm.

VI. CONCLUSION

In this paper, we propose an evolution method based on cluster analysis to improve the effectiveness of test suite reduction. The K-means algorithm is used to cluster test cases, and the genetic algorithm is utilized to reduce the test suite with the failure detection rate, code coverage, and execution time as optimization targets. The experimental evaluation shows that the proposed method takes advantage of removing redundant test cases and detecting failures, while keeping comparable testing efficiency compared with the other three methods. At the same time, the experiments also prove that our method has the best missing failure rate and code coverage rate of the test case.

In the future, we will conduct a series of test suite reduction research from machine learning technology, to reduce the cost of regression testing and ensure software quality.

REFERENCES

- [1] C. Xia, Y. Zhang, L. Wan, Y. Song, N. Xiao, and B. Guo, "Test data generation of path coverage based on negative selection genetic algorithm," *Acta Electronica Sinica*, vol. 47, no. 12, p. 2630, 2019.
- [2] D. Pradhan, S. Wang, S. Ali, T. Yue, and M. Liaaen, "Employing rule mining and multi-objective search for dynamic test case prioritization," *J. Syst. Softw.*, vol. 153, pp. 86–104, Jul. 2019.
- [3] C. Xia, X. Wang, and Y. Zhang, "Test case prioritization based on multi-objective optimization," *Comput. Sci.*, vol. 47, no. 6, p. 38, 2020.
- [4] G. Rothermel and M. J. Harrold, "Analyzing regression test selection techniques," *IEEE Trans. Softw. Eng.*, vol. 22, no. 8, pp. 529–551, Aug. 1996.
- [5] G. Rothermel, M. J. Harrold, J. Ostrin, and C. Hong, "An empirical study of the effects of minimization on the fault detection capabilities of test suites," in *Proc. Int. Conf. Softw. Maintenance*, Nov. 1998, pp. 34–43.
- [6] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold, "Prioritizing test cases for regression testing," *IEEE Trans. Softw. Eng.*, vol. 27, no. 10, pp. 929–948, Oct. 2001.
- [7] M. A. Alipour, A. Shi, R. Gopinath, D. Marinov, and A. Groce, "Evaluating non-adequate test-case reduction," in *Proc. 31st IEEE/ACM Int. Conf. Automated Softw. Eng.*, Aug. 2016, pp. 16–26.
- [8] S. Yoo and M. Harman, "Using hybrid algorithm for Pareto efficient multi-objective test suite minimisation," *J. Syst. Softw.*, vol. 83, no. 4, pp. 689–701, Apr. 2010.
- [9] W. Zheng, R. M. Hierons, M. Li, X. Liu, and V. Vinciotti, "Multi-objective optimisation for regression testing," *Inf. Sci.*, vols. 334–335, pp. 1–16, Mar. 2016.
- [10] C. Zhang, Z. Chen, Z. Zhao, S. Yan, J. Zhang, and B. Xu, "An improved regression test selection technique by clustering execution profiles," in *Proc. 10th Int. Conf. Qual. Softw.*, Jul. 2010, pp. 171–179.
- [11] S. Chen, Z. Chen, Z. Zhao, B. Xu, and Y. Feng, "Using semi-supervised clustering to improve regression test selection techniques," in *Proc. 4th IEEE Int. Conf. Softw. Test., Verification Validation*, Mar. 2011, pp. 1–10.
- [12] Y. Wang, R. Gao, Z. Chen, W. E. Wong, and B. Luo, "WAS: A weighted attribute-based strategy for cluster test selection," *J. Syst. Softw.*, vol. 98, pp. 44–58, Dec. 2014.
- [13] X. Zhao, Z. Wang, X. Fan, and Z. Wang, "A clustering-Bayesian network based approach for test case prioritization," in *Proc. IEEE 39th Annu. Comput. Appl. Conf.*, Jul. 2015, pp. 542–547.
- [14] H. Pei, B. Yin, K.-Y. Cai, and M. Xie, "A distance-based dynamic random testing with test case clustering," in *Proc. IEEE 19th Int. Conf. Softw. Qual., Rel. Secur. (QRS)*, Jul. 2019, pp. 46–53.
- [15] A. Saxena, M. Prasad, A. Gupta, N. Bharill, O. P. Patel, A. Tiwari, M. J. Er, W. Ding, and C.-T. Lin, "A review of clustering techniques and developments," *Neurocomputing*, vol. 267, pp. 664–681, Dec. 2017.
- [16] L. Zhang, D. Hao, L. Zhang, G. Rothermel, and H. Mei, "Bridging the gap between the total and additional test-case prioritization strategies," in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, May 2013, pp. 192–201.
- [17] H. Zhong, L. Zhang, and H. Mei, "An experimental study of four typical test suite reduction techniques," *Inf. Softw. Technol.*, vol. 50, no. 6, pp. 534–546, May 2008.
- [18] L. Zhang, D. Marinov, L. Zhang, and S. Khurshid, "An empirical study of JUnit test-suite reduction," in *Proc. IEEE 22nd Int. Symp. Softw. Rel. Eng.*, Nov. 2011, pp. 170–179.
- [19] Y. Lou, D. Hao, and L. Zhang, "Mutation-based test-case prioritization in software evolution," in *Proc. IEEE 26th Int. Symp. Softw. Rel. Eng. (ISSRE)*, Nov. 2015, pp. 46–57.
- [20] A. Shi, A. Gyori, M. Gligoric, A. Zaytsev, and D. Marinov, "Balancing trade-offs in test-suite reduction," in *Proc. 22nd ACM SIGSOFT Int. Symp. Found. Softw. Eng. FSE*, 2014, pp. 246–256.
- [21] J. Chen, Y. Lou, L. Zhang, J. Zhou, X. Wang, D. Hao, and L. Zhang, "Optimizing test prioritization via test distribution analysis," in *Proc. 26th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Oct. 2018, pp. 656–667.
- [22] R. Mukherjee and K. S. Patnaik, "A survey on different approaches for software test case prioritization," *J. King Saud Univ. Comput. Inf. Sci.*, 2018, doi: [10.1016/j.jksuci.2018.09.005](https://doi.org/10.1016/j.jksuci.2018.09.005).
- [23] Y. Lou, J. Chen, L. Zhang, and D. Hao, "A survey on regression test-case prioritization," in *Advances in Computers*, vol. 113. Amsterdam, The Netherlands: Elsevier, 2019, pp. 1–46.
- [24] X. Chunyan, W. Xingya, Z. Yan, and Y. Hao, "Using genetic algorithm to augment test data for penalty prediction," *Int. J. Performability Eng.*, vol. 16, no. 7, p. 1078, 2020.
- [25] M. Li, T. Chen, and X. Yao, "How to evaluate solutions in Pareto-based search-based software engineering? A critical review and methodological guidance," *IEEE Trans. Softw. Eng.*, early access, Nov. 9, 2020, doi: [10.1109/TSE.2020.3036108](https://doi.org/10.1109/TSE.2020.3036108).
- [26] O. Ö. Özener and H. Sözer, "An effective formulation of the multi-criteria test suite minimization problem," *J. Syst. Softw.*, vol. 168, Oct. 2020, Art. no. 110632.
- [27] G. Chou-Guang, L. Du, and X. Hao, "Research of software testing case reduction algorithm based on k means," *Microelectron. Comput.*, vol. 33, no. 5, pp. 133–141, May 2016.
- [28] J. Chen, L. Zhu, T. Y. Chen, D. Towey, F.-C. Kuo, R. Huang, and Y. Guo, "Test case prioritization for object-oriented software: An adaptive random sequence approach based on clustering," *J. Syst. Softw.*, vol. 135, pp. 107–125, Jan. 2018.
- [29] S. Yoo and M. Harman, "Regression testing minimization, selection and prioritization: A survey," *Softw. Test., Verification Rel.*, vol. 22, no. 2, pp. 67–120, Mar. 2012.
- [30] M. Harder, J. Mellen, and M. D. Ernst, "Improving test suites via operational abstraction," in *Proc. 25th Int. Conf. Softw. Eng.*, May 2003, pp. 60–71.
- [31] S. Mcmaster and A. Memon, "Call-stack coverage for GUI test suite reduction," *IEEE Trans. Softw. Eng.*, vol. 34, no. 1, pp. 99–115, Jan. 2008.

- [32] J. Campos and R. Abreu, "Leveraging a constraint solver for minimizing test suites," in *Proc. 13th Int. Conf. Qual. Softw.*, Jul. 2013, pp. 253–259.
- [33] A. Gotlieb and D. Marijan, "FLOWER: Optimal test suite reduction as a network maximum flow," in *Proc. Int. Symp. Softw. Test. Anal. ISSTA*, 2014, pp. 171–180.
- [34] S. Arlt, A. Podelski, and M. Wehrle, "Reducing GUI test suites via program slicing," in *Proc. Int. Symp. Softw. Test. Anal. ISSTA*, 2014, pp. 270–281.
- [35] C.-T. Lin, K.-W. Tang, and G. M. Kapfhammer, "Test suite reduction methods that decrease regression testing costs by identifying irreplaceable tests," *Inf. Softw. Technol.*, vol. 56, no. 10, pp. 1322–1344, Oct. 2014.
- [36] S. Wang, S. Ali, and A. Gotlieb, "Cost-effective test suite minimization in product lines using search techniques," *J. Syst. Softw.*, vol. 103, pp. 370–391, May 2015.
- [37] M. Mussa and F. Khendek, "Model-based test cases reuse and optimization," in *Advances in Computers*, vol. 113. Amsterdam, The Netherlands: Elsevier, 2019, pp. 47–87.
- [38] A. Groce, M. A. Alipour, C. Zhang, Y. Chen, and J. Regehr, "Cause reduction for quick testing," in *Proc. IEEE 7th Int. Conf. Softw. Test., Verification Validation*, Mar. 2014, pp. 243–252.
- [39] X.-Y. Ma, Z.-F. Ha, B.-K. Sheng, and C.-Q. Ye, "A genetic algorithm for test-suite reduction," in *Proc. IEEE Int. Conf. Syst., Man Cybern.*, Oct. 2005, pp. 133–139.
- [40] N. Gupta, A. Sharma, and M. K. Pachariya, "Multi-objective test suite optimization for detection and localization of software faults," *J. King Saud Univ. Comput. Inf. Sci.*, 2020, doi: [10.1016/j.jksuci.2020.01.009](https://doi.org/10.1016/j.jksuci.2020.01.009).
- [41] J. Costa, "A multi-objective optimization approach to test suite reduction," M.S. thesis, Faculdade de Engenharia da Universidade do Porto, Mestrado Integrado em Engenharia Informática e Computação, Porto, Portugal, 2015.
- [42] X.-M. Cheng, Q.-H. Yang, Y.-P. Zhai, and W. CHEN, "Test case selection technique base on semi-supervised clustering method," *Comput Sci*, vol. 45, no. 1, pp. 249–254, 2018.
- [43] H. Do, "Recent advances in regression testing techniques," in *Advances in Computers*, vol. 103. Amsterdam, The Netherlands: Elsevier, 2016, pp. 53–77.



CHUNYAN XIA received the B.S. and M.S. degrees from the Changchun University of Science and Technology, in 2003 and 2007, respectively. She is currently the Associate Professor with Mudanjiang Normal University. As the first author, she published more than ten articles, EI included four articles. She presided over one project of Heilongjiang Provincial Natural Science Foundation of China. Her research interests include search-based software engineering and data mining technology.



Her research interest includes search-based software engineering.

YAN ZHANG received the Ph.D. degree from the China University of Mining and Technology, in 2012. She is currently a Professor with Mudanjiang Normal University. As the first author, she published more than ten articles, SCI and EI included more than eight articles. She received one national invention patent. She presided over one project of Heilongjiang Provincial Natural Science Foundation of China and one youth backbone support project of Heilongjiang Province.



He received nine national invention patents. He presided over one sub-project of the National Key Research and Development Program, one youth project of the Natural Science Foundation, one youth project of the Natural Science Foundation of Jiangsu Province, one project of the National Postdoctoral Foundation, and eight lateral software evaluation projects of the army. His research interests include uncertainty test output verification technology and test quality assessment technology. He also received the CCF-CTC Software Testing Innovation Award. He awarded the Excellent Doctoral Dissertation Award of PLA University of Science and Technology, in 2017, the Excellent Doctoral Dissertation Award of Jiangsu Computer Society, in 2017, and the Excellent Doctoral Dissertation of China Accusation and Control Society, in 2018.

• • •